U21ADP05 EXPLORATORY DATA ANALYSIS AND VISULIZATION

Case Study: Data Exploration and Visualization of Breast Cancer Dataset

Name: ABINAYA S

Roll No.: 24D001

Department: ARTIFICAL INTELLIGENCE AND DATA SCIENCE

Date of Submission: 15 October 2026

Abstract

Early and accurate classification of breast masses as benign or malignant supports timely treatment. This study performs exploratory data analysis, statistical analysis and visualization of the Breast Cancer Wisconsin (Diagnostic) dataset, which contains 569 patients described by 30 numeric cell-nucleus features (357 benign, 212 malignant). The data had no missing values or duplicates; 171 rows contained IQR outliers and were winsorized. Twelve visualizations covered distributions, correlations, group differences, PCA and probability models. Size and concavity features separated the classes strongly (Cohen's d up to 2.6), while 21 feature pairs were highly collinear. A PyTorch multilayer perceptron tuned by grid search reached 97.7% test accuracy, 100% recall and AUC 1.000, comparable to logistic regression and random forest baselines.

1. Introduction and Objective

Breast cancer is one of the most common cancers worldwide, and the first diagnostic step for a palpable mass is often a fine-needle aspiration (FNA). A pathologist examines the cells under a microscope, a process that is subjective and time consuming. Computer-aided analysis of digitized FNA images can provide a consistent second opinion. Wolberg, Street and Mangasarian extracted quantitative nuclear features from such images, producing the dataset analysed here (Street et al., 1993; Wolberg et al., 1995).

Objective. To apply data-visualization, statistical and exploratory techniques to this real-world medical dataset, derive insight into which cell characteristics distinguish malignant from benign masses, and evaluate a deep-learning classifier built on the cleaned data.

Specific goals:

- Profile the dataset: structure, missing values, duplicates, outliers and distributions.
- Identify univariate, bivariate and multivariate patterns using at least five (here twelve) visualizations.
- Fit and interpret probability distributions (continuous and discrete).
- Train, tune and evaluate a PyTorch multilayer perceptron (MLP) and compare it with two classical baselines.
- Analyse feature importance and discuss limitations and future scope.

Visualization pipeline (the seven stages)

Stage	How it was applied in this project
1. Acquire	Dataset obtained from the UCI Machine Learning Repository (Wolberg et al., 1995); loaded via scikit-learn and exported to data/breast_cancer_wisconsin.csv.
2. Parse	Fields identified (30 numeric features + diagnosis label); data types verified; target encoded Malignant = 1, Benign = 0.
3. Filter	Duplicates and missing values checked (none found); outliers flagged with the 1.5×IQR rule and winsorized at the 1st/99th percentile.
4. Mine	Descriptive statistics, skewness, Shapiro-Wilk, KS tests, Welch t-test, Mann-Whitney U, Cohen's d, correlation, PCA.
5. Represent	Chart type chosen per question: bars/pie (class balance), histogram/KDE (distribution), boxplot/violin (spread), heatmap (correlation), scatter/pairplot (relationships), PCA plot.
6. Refine	Consistent colour-blind-safe palette (blue = benign, orange = malignant), titles, axis labels, legends, standardized scales.
7. Interact	Interactive Plotly explorer (figures/interactive_explorer.html) with feature drop-downs, hover details, zoom and legend toggling.

2. Dataset Description

Source: UCI Machine Learning Repository – Breast Cancer Wisconsin (Diagnostic) (https://doi.org/10.24432/C5DW2B); also distributed with scikit-learn.

Size: 569 rows × 31 columns (30 features + 1 label).

Type: Numerical, tabular. Each row is one patient; features are computed from a digitized image of an FNA of a breast mass

Base measurement	Meaning
radius	Mean distance from the centre to points on the nucleus boundary
texture	Standard deviation of grey-scale values
perimeter / area	Size of the nucleus boundary / region
smoothness	Local variation in radius lengths
compactness	perimeter² / area − 1.0
concavity	Severity of concave portions of the contour
concave points	Number of concave portions of the contour
symmetry	Symmetry of the nucleus
fractal dimension	"Coastline approximation" − 1 (boundary irregularity)

Feature	Mean	Std	Min	Median	Max
mean radius	14.127	3.524	6.981	13.370	28.110
mean texture	19.290	4.301	9.710	18.840	39.280
mean area	654.889	351.914	143.500	551.100	2501.000
mean smoothness	0.096	0.014	0.053	0.096	0.163
mean concavity	0.089	0.080	0.000	0.061	0.427
area error	40.337	45.491	6.802	24.530	542.200
worst area	880.583	569.357	185.200	686.500	4254.000

3. EDA and Preprocessing

3.1 Methods

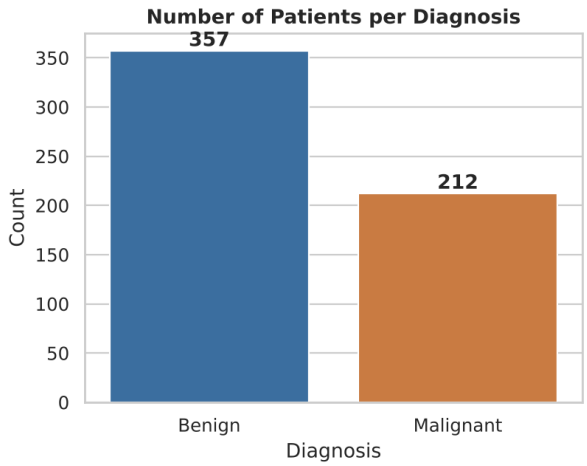
Step	Method	Finding / action
Missing values	isna().sum()	0 missing values – no imputation needed
Duplicates	duplicated().sum()	0 duplicate rows – nothing removed
Outliers	1.5×IQR rule on each feature	608 flagged values in 171 of 569 rows (30.1%); most in "area error" (65), "radius error" (38) and "perimeter error" (38)
Outlier treatment	Winsorize at 1st/99th percentile (bounds from training set only)	Extreme values retained but limited, because large tumours are real and clinically meaningful
Skewness	Sample skewness	22 of 30 features have |skew| > 1 (right-skewed)
Encoding	Label encoding	Malignant = 1 (positive class), Benign = 0
Scaling	Z-score standardization	Mean 0, std 1 per feature; essential for neural-network training
Splitting	Stratified split, random_state = 42	Train (70%) / Validation (15%) / Test (15%)

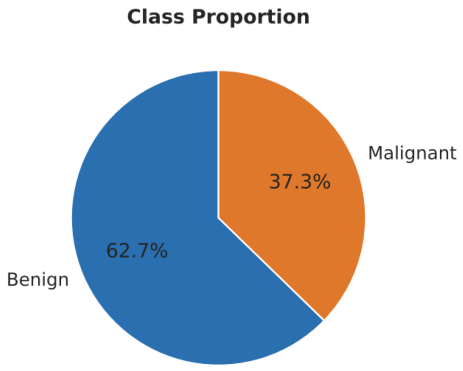
3.2 Insights

- **Clean but not trivial.** The data are complete, but about three in ten patients have at least one outlying value and most features are right-skewed, so scaling and outlier handling still matter.

- **Moderate class imbalance.** 62.7% benign versus 37.3% malignant; stratified splitting preserves this ratio in all three sets.

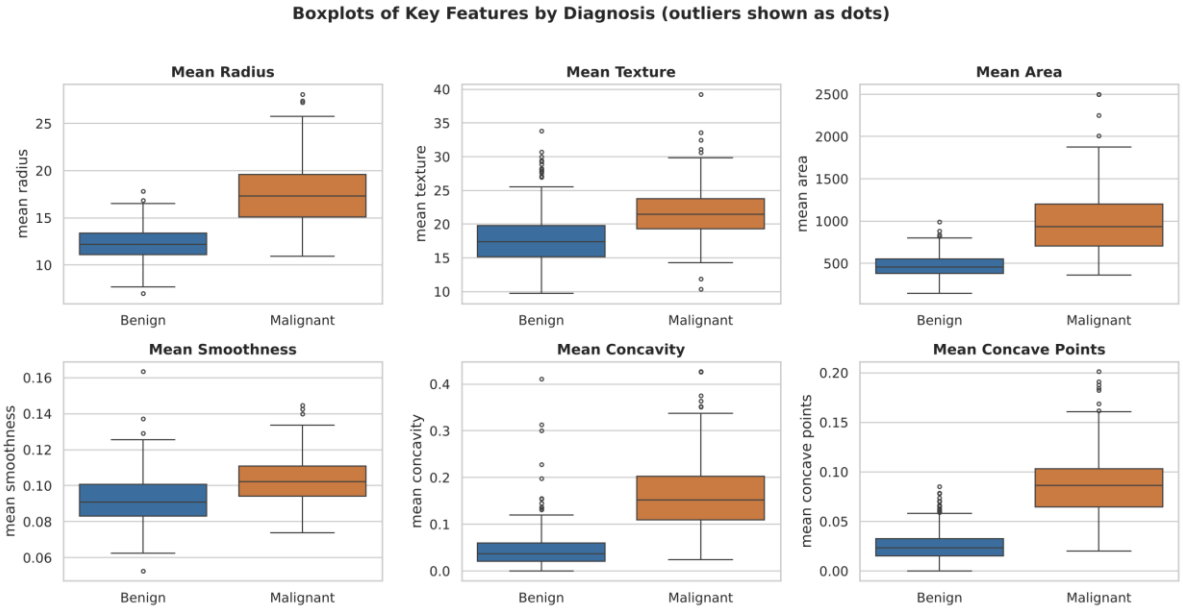
4. Data Visualization





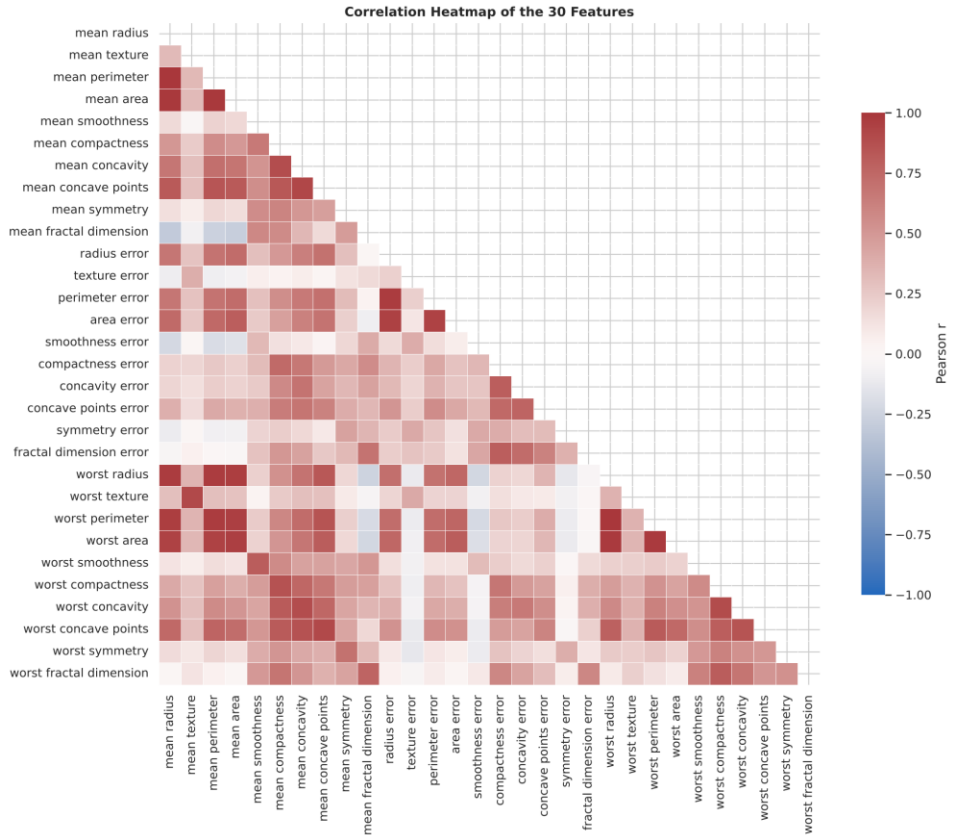
What it shows: The number and share of benign and malignant patients.

Why it was created: To check class balance before modelling, because imbalance affects the choice of metrics and splitting.



What it shows: Median, quartiles and outliers of each feature in each class.

Why it was created: To compare spread and locate outliers visually.



What it shows: Pairwise linear correlation between all features.

Why it was created: Multivariate analysis: to detect redundancy and groups of related features.

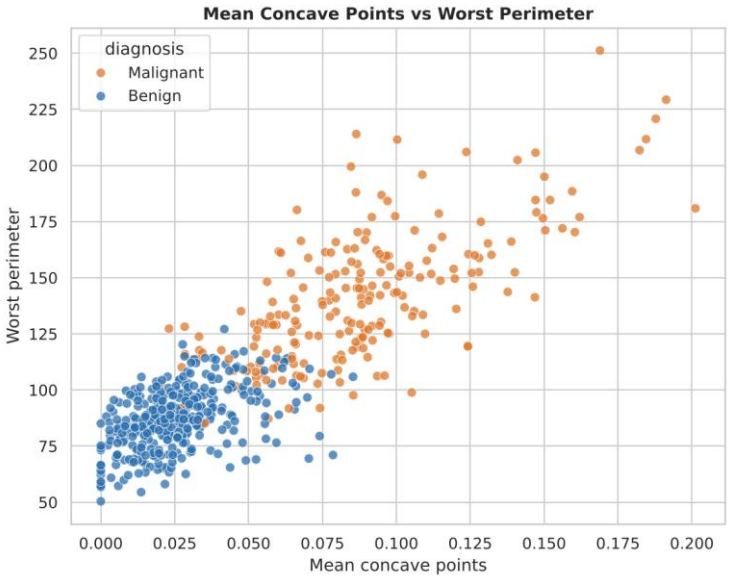


Figure 5. Scatter plot: mean concave points vs worst perimeter.

What it shows: Two highly discriminating features plotted against each other, coloured by diagnosis.

Why it was created: Bivariate analysis: to see whether two features alone can separate the classes.

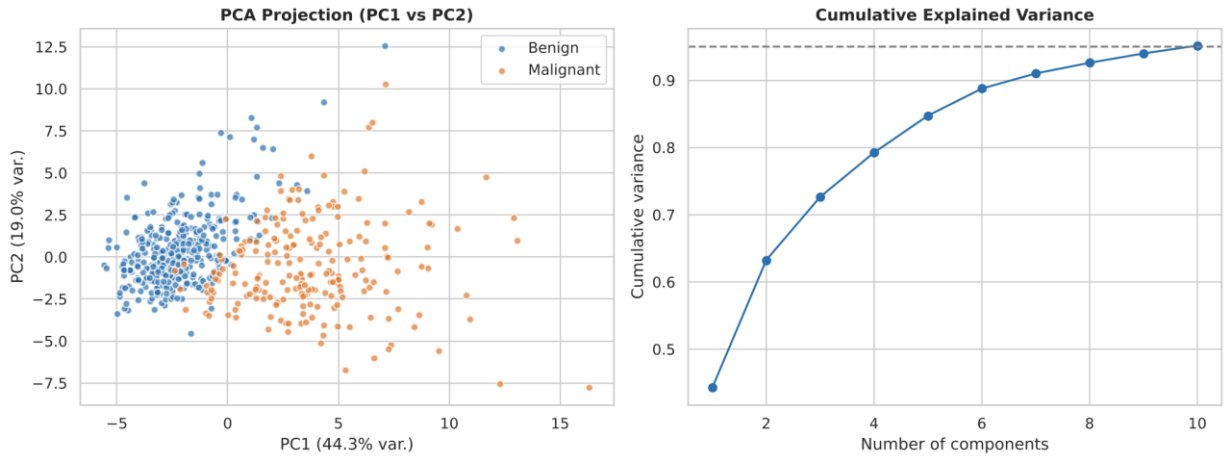


Figure 11. PCA projection (PC1 vs PC2) and cumulative explained variance.

What it shows: The 30 standardized features reduced to two principal components, and the variance explained as components are added.

Why it was created: Multivariate analysis: to see whether the high-dimensional data have simple structure and how redundant the features are.

5. Deep Learning Model

5.1 Architecture

A fully connected multilayer perceptron (MLP) was implemented in PyTorch (Paszke et al., 2019). The selected architecture is 30 -> 64 -> 32 -> 1 with 4,289 trainable parameters:

Layer	Details
Input	30 standardized features
Hidden 1	Linear(30→64) → BatchNorm → ReLU → Dropout(0.3)
Hidden 2	Linear(64→32) → BatchNorm → ReLU → Dropout(0.3)
Output	Linear(32→1) logit; sigmoid applied inside the loss function, probability of malignancy at inference

5.2 Training parameters

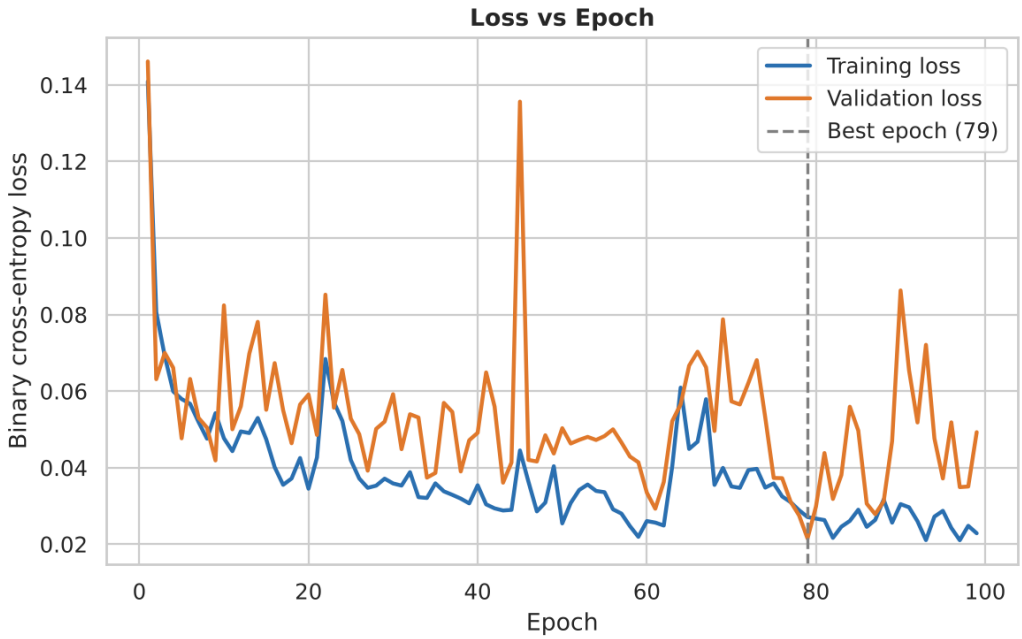
Parameter	Value
Optimizer	Adam
Loss	Binary cross-entropy with logits
Batch size	32
Weight decay (L2)	0.0001
Maximum epochs	200
Early stopping	Patience 20 on validation loss; best weights restored
Epochs run / best epoch	99 / 79
Decision threshold	0.5
Random seed	42
Hardware	CPU

5.3 Hyperparameter tuning

A basic grid search was run over hidden-layer structure (16; 32-16; 64-32), learning rate (0.001, 0.01) and dropout (0.0, 0.3), giving 12 configurations. Each was trained with early stopping and ranked by validation loss. Selected: hidden layers (64, 32), learning rate 0.01, dropout 0.3. All 12 configurations reached 97.7%–100.0% validation accuracy, and with only 86 validation patients one patient equals about 1.2 percentage points, so the differences between configurations are small relative to noise. Validation loss was used because it is more sensitive than accuracy.

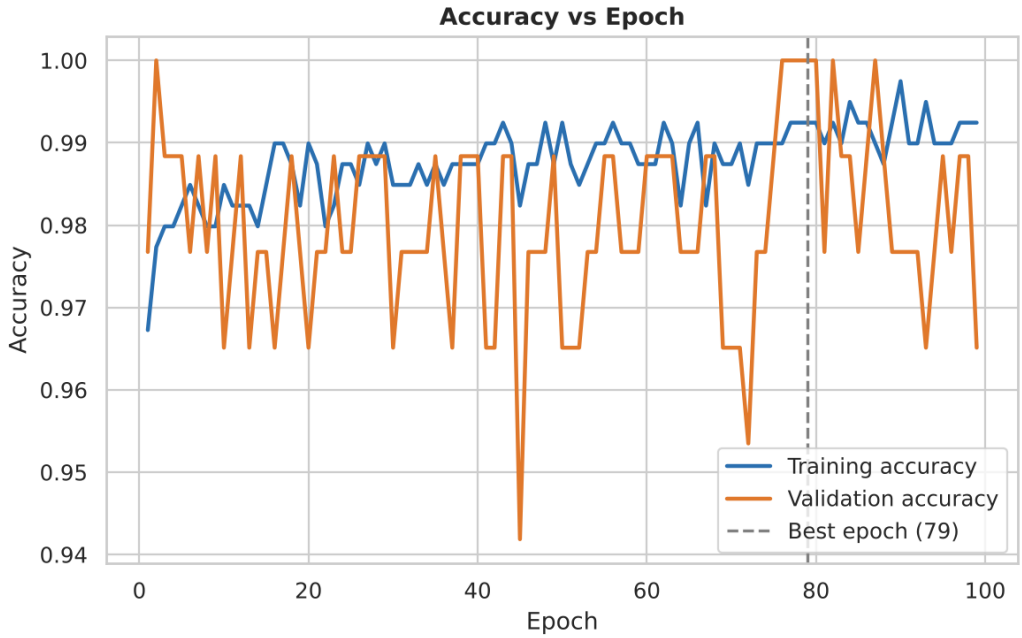
6. Result Visualization and Interpretation

Metric	Value
Accuracy	0.9767
Precision	0.9412
Recall (sensitivity)	1.0000
F1-score	0.9697
ROC-AUC	1.0000



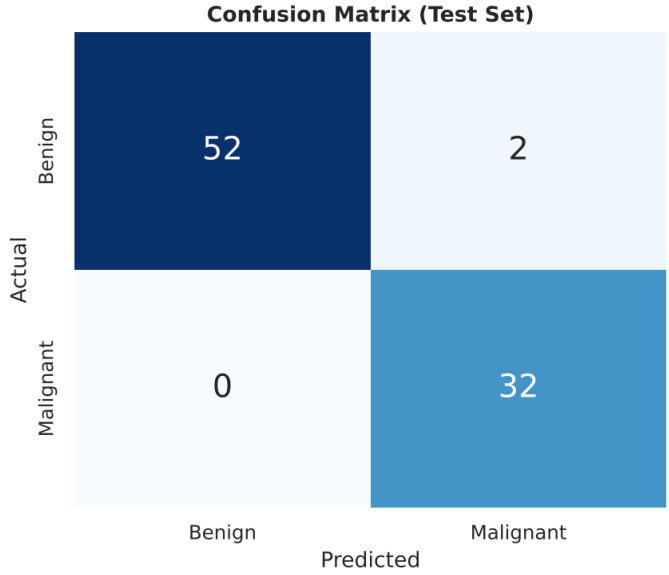
What it shows: Binary cross-entropy on training and validation data over the epochs.

Why it was created: To check convergence and overfitting.



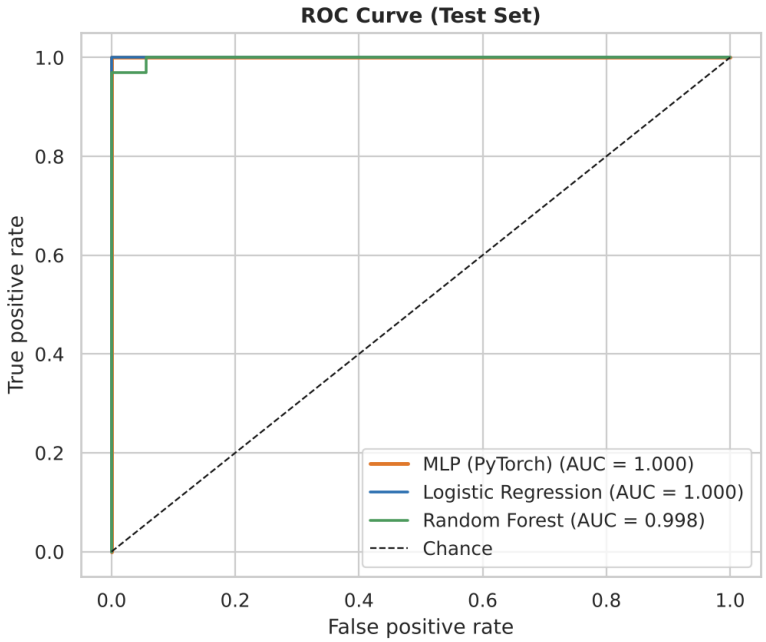
What it shows: Fraction of correct predictions on training and validation data per epoch.

Why it was created: To complement the loss curve with an interpretable metric.



What it shows: Counts of correct and incorrect predictions per class.

Why it was created: Accuracy hides the type of error; in cancer screening a missed malignancy costs more than a false alarm.



What it shows: True-positive rate against false-positive rate over all decision thresholds, with the area under the curve (AUC).

Why it was created: To evaluate ranking quality independent of the 0.5 threshold, and to compare with baselines.

7. Conclusion and Future Scope

7.1 What was learned

- **Dataset.** The data are complete and clean, but skewed (22 of 30 features) and highly redundant (21 feature pairs with $|r| > 0.9$).
- **Key patterns.** Malignant nuclei are larger and more irregular. The strongest discriminators are worst concave points ($d = 2.6$), worst perimeter (2.37), mean concave points (2.33) and worst radius (2.32). Worst-case ('worst') features were generally more discriminative than the mean values of the same measurement.
- **Distributions.** Size features are right-skewed and closer to Lognormal than Normal; the class label follows a Bernoulli/Binomial model with $p \approx 0.37$.
- **Model performance.** The MLP reached 97.7% accuracy, 100% recall, precision 0.941 and AUC 1.000 on the test set; across 10 seeds, accuracy averaged 99.2%. It matched, but did not beat, logistic regression.

7.2 Limitations

- Only 569 patients from a single source; no external validation set, so generalization to other hospitals and imaging devices is unknown.
- Features are pre-extracted from images by an earlier algorithm, so errors in segmentation are not visible.
- A single stratified split was used; cross-validation would give tighter estimates.
- p-values were not corrected for multiple comparisons across 30 features.
- This is an educational analysis, not a clinical tool.

7.3 Future scope

- Use stratified k-fold cross-validation with confidence intervals for every model, and a formal test such as McNemar's for model comparison.
- Reduce collinearity with PCA or by dropping redundant features, then use SHAP values for more reliable importance.
- Calibrate the predicted probabilities and choose the threshold according to the cost of false negatives.
- Validate on an independent dataset and build a small web application for demonstration.

8. References

Cohen, J. (1988). Statistical power analysis for the behavioral sciences (2nd ed.). Lawrence Erlbaum Associates.

Fry, B. (2007). Visualizing data: Exploring and explaining data with the Processing environment. O'Reilly Media.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. Proceedings of IS&T/SPIE International Symposium on Electronic Imaging, 1905, 861–870.

Waskom, M. L. (2021). seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic) [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B

9. Appendix – Code

The complete, commented source code is in the GitHub repository. Key files:

Code extract: MLP definition and training loop

```
class MLP(nn.Module):
    """Fully-connected network: [Linear -> BatchNorm -> ReLU -> Dropout] x N
-> Linear(1)."""
    def __init__(self, in_dim, hidden, dropout):
        super().__init__()
        layers, prev = [], in_dim
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ReLU(),
nn.Dropout(dropout)]
            prev = h
        layers.append(nn.Linear(prev, 1))          # logit output (sigmoid
inside the loss)
        self.net = nn.Sequential(*layers)
    def forward(self, x):
        return self.net(x).squeeze(1)
def to_t(a, dtype=torch.float32):
    return torch.tensor(np.asarray(a), dtype=dtype)
Ttr, Tva, Tte = to_t(Xtr), to_t(Xva), to_t(Xte)
```

```
ytr_t, yva_t, yte_t = to_t(y_train), to_t(y_val), to_t(y_test)
def train_model(hidden, lr, dropout, weight_decay=1e-4, max_epochs=200,
patience=20, batch=32, seed=SEED, verbose=False):
"""Train with Adam + BCE-with-logits and early stopping on validation
loss."""
    torch.manual_seed(seed)
    model = MLP(Xtr.shape[1], hidden, dropout)
    opt = torch.optim.Adam(model.parameters(), lr=lr,
weight_decay=weight_decay)
    lossf = nn.BCEWithLogitsLoss()
    hist = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
    best, best_state, wait = np.inf, None, 0
    n = len(Ttr)
    for ep in range(max_epochs):
        model.train()
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            if len(idx) < 2:
                continue
            opt.zero_grad()
            loss = lossf(model(Ttr[idx]), ytr_t[idx])
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            ltr, lva = model(Ttr), model(Tva)
            hist["train_loss"].append(lossf(ltr, ytr_t).item())
            hist["val_loss"].append(lossf(lva, yva_t).item())
            hist["train_acc"].append(((ltr > 0).float() ==
ytr_t).float().mean().item())
            hist["val_acc"].append(((lva > 0).float() ==
yva_t).float().mean().item())
        if hist["val_loss"][-1] < best - 1e-4:
            best, wait = hist["val_loss"][-1], 0
            best_state = {k: v.clone() for k, v in
model.state_dict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    model.eval()
    hist["best_epoch"] = int(np.argmin(hist["val_loss"]) + 1)
    return model, hist
```